

Dionysus 下一阶段三阶段策划文档

文档性质：策划 / 方向性文档（非执行计划）
编写方式：多角色协作讨论纪要 + 现状诊断 + 阶段方向建议
适用仓库：FuyuuKuSakura/DionysusC（Dionysus v0.2.0）
目标：为后续三个环节——PR 合并、仓库整理、前端/移动端改造——提供清晰的问题清单、方向共识和待决策事项。

1. 编写说明

本文档的核心使命是“把问题聊清楚、把方向定下来”。文中通过模拟 **架构师、软件工程专家、前端设计艺术家、产品专家** 四个角色进行多轮讨论，并结合用户反馈逐步收敛，最终形成三阶段的策划方向、关键原则和剩余待决策事项。

在后续进入每个阶段时，可以基于本文档再拆解出具体的执行计划、任务列表和验收标准。本文档本身不替代执行计划。

2. 项目概况

Dionysus（代号“苹果派”）是一个带 Live2D 角色陪伴的 Coding Agent 客户端：

- 桌面端：**Electron + PyInstaller 打包的 Python FastAPI 后端。
- 浏览器/移动端：**React 18 + Vite + Tailwind CSS，通过 WebSocket 与后端通信。
- 已接入 Agent CLI：**Kimi Code CLI、Claude Code CLI、Codex CLI、OpenCode CLI。
- 当前公开 PR：**feat: add CodeBuddy Code CLI adapter and collapsible thinking section，计划新增 CodeBuddy 适配器与可折叠“思考过程”UI。

3. 现状与问题诊断

3.1 硬编码本机路径（高风险 / 必须处理）

仓库中发现了多处写死的本机路径，主要包括：

位置	问题示例	影响
backend/config/server.yaml	working_dir: "/Users/fuyuuku/ACP_AGENT2/test_workspace"	其他开发者无法直接运行；泄露本机

位置	问题示例	影响
		目录结构
scripts/qa_*.py 、 scripts/test_adapters.py	输出目录写死为 /Users/fuyuuku/ACP_AGENT2/...	QA 脚本不具备可移植性
frontend/tests/*.py	截图路径写死	测试脚本在其他机器 / CI 上失效
README.md	示例 working_dir: "/Users/fuyuuku/projects"	公开仓库给人“半成品”印象
backend/dionysus_server/session/manager.py	提示语中示例路径 /cd /Users/fuyuuku/project	遗留痕迹

处理原则：所有路径应改为相对路径、环境变量或命令行参数，默认行为应能在 clone 仓库后直接运行；发布产品时工作目录应落在用户数据区而非应用包内。

3.2 目录与资产结构需要梳理

- **Live2D 模型重复：**仓库根目录存在 kal'tsit_live2d/ 、 exusiai_live2d/ （源模型），同时 backend/config/personas/live2d/ 下存在运行时 copy，关系不清晰。
- **Electron 入口疑似重复：**仓库根目录有 electron/ ， frontend/electron/ 也有主进程。需要确认哪个是实际打包入口，清理废弃版本。
- **脚本目录较乱：**QA 脚本散落在 scripts/ 根目录，缺少分类。
- **输出目录未统一：**QA 截图、报告散落在 qa_screenshots/ 、 qa_reports/ 、 frontend_screenshot_*.png 等，需要统一纳入可忽略目录。

3.3 配置与环境变量不一致

- scripts/launcher.py 使用 DIONYSUS_SERVER__HOST （全大写），而 pydantic-settings 的 env_prefix 配置为 Dionysus_ 。在 Linux 等大小写敏感的环境中可能导致环境变量无法被识别。
- 后端已有 Dionysus_CONFIG_DIR 、 Dionysus_DATA_DIR 的设计，但前端/脚本侧尚未完全对齐。

3.4 开发文档缺口

- README 目前偏“用户安装指南”，缺少面向开发者的模块说明。
- 没有清晰的“功能 ↔ 文件 ↔ 接口”映射表。
- 缺少本地开发、测试、发布的标准化说明。

3.5 移动端现状

- 前端已有若干 Mobile* 组件（MobileCompanionBar、MobileCompanionDrawer、MobileResourcePanel、MobileSessionListHeader），说明移动端曾被考虑。
- 但这些组件更像是补丁式添加，缺乏统一的设计语言和响应式骨架。
- Live2D 在移动端如何呈现（常驻、折叠、隐藏、性能优化）尚未形成一致方案。

4. 待合并 PR 概要

PR #1: feat: add CodeBuddy Code CLI adapter and collapsible thinking section（贡献者：AblazeGHR）

主要改动：

1. 新增 backend/dionysus_server/agent_adapters/strategies/codebuddy.py，实现 CodeBuddy CLI 的 stream-json 解析。
2. 在 registry.py / strategies/__init__.py 中注册 CodeBuddy。
3. 给 AgentStreamPayload 增加 is_thinking 字段，用于标识思考内容。
4. 前端新增 ThinkingSection 组件，在 AgentMessage / StreamingStatusBox 中可折叠展示思考过程。
5. chatStore 增加 addThinkingChunk / addThinkingChunkToSession。
6. server.yaml 中各 adapter 的 working_dir 改为相对路径 ../workspace。
7. 补充 httpx、qrcode[pil]、python-multipart 等依赖。

合并关注点：

- PR 本身已将 working_dir 改为相对路径，与仓库整理目标一致。
- CodeBuddy 的策略需要单元测试覆盖主要 JSON 事件类型。
- 思考区 UI 需要保证旧 adapter（无 thinking）不回显空区域。
- 合并后需对所有 adapter 做最小 smoke 验证。

5. 多角色协作讨论纪要

角色设定

- 架构师（A）：关注系统结构、可维护性、配置/数据隔离。
- 软件工程专家（SE）：关注代码质量、测试、依赖、构建、可移植性。
- 前端设计艺术家（FDA）：关注视觉系统、交互体验、移动端呈现、动效。
- 产品专家（PM）：关注用户价值、版本节奏、PR 风险、文档可读性。

Round 1: 各自抛出核心关注点

A: 我最先关注的是仓库的“可公开维护性”。目前仓库里存在大量 `/Users/fuyuuku/...` 的硬编码路径，不仅让其他开发者跑不起来，也暴露了本机信息。其次，目录结构上有三处让我困惑：1. 根目录的 `kal'tsit_live2d/`、`exusiai_live2d/` 与 `backend/config/personas/live2d/` 是什么关系？2. 根目录的 `electron/` 和 `frontend/electron/` 哪个才是打包真正用的？3. 配置目录与运行时数据目录是否应通过 `Dionysus_CONFIG_DIR` / `Dionysus_DATA_DIR` 明确隔离？

我的主张是：先建立一套“源码 / 配置 / 运行时数据”清晰分离的目录契约，再谈后续功能。

SE: 我补充工程实现层面的问题。硬编码路径只是表象，更深的问题是：- `generic_cli.py` 拿到 `working_dir` 后没有做任何“相对路径解析”，如果用户给的是相对路径，实际 `cwd` 会取决于启动 `uvicorn` 的目录，这是不可预期的。- `scripts/launcher.py` 的环境变量前缀 `DIONYSUS_SERVER_HOST` 和 `pydantic-settings` 的 `env_prefix="Dionysus_"` 不一致，Linux 上会失效。- QA 脚本和测试脚本都假设项目在 `/Users/fuyuuku/ACP_AGENT2`，CI 和其他机器无法复用。- 目前 `pyproject.toml` 里已有 `ruff/mypy` 配置，但缺少严格执行的流程。

我希望在整理阶段能把“路径解析规则、环境变量命名、脚本可移植性、基础 lint”这四件事固定下来。

FDA: 从设计角度看，前端目前处于“能用但不统一”的状态。Tailwind class 大量硬编码，主题由 YAML 驱动但和组件代码没有形成统一 Token。移动端虽然有几个 `Mobile*` 组件，但感觉是补丁式加入，没有整体布局策略。

我建议：- 建立一套设计 Token（颜色、间距、圆角、字体、动效）。- 把 Tailwind 配置和主题 YAML 对齐，避免“两套皮肤”。- 移动端以“会话优先”重新排布：底部输入栏、可折叠角色抽屉、全屏聊天、Live2D 可小窗或隐藏。- 桌面端在视觉上需要一次刷新，但结构保持“左右分栏 + 角色在右”。

PM: 大家要记住最终交付的是“一个公开可维护、能合并 PR、能在手机上用的产品”。

我建议：1. 先让仓库在 GitHub 上达到“可公开维护”的底线：没有隐私路径、有文档、有标准结构。2. 把 PR 平滑合并，不要影响现有 Kimi/Claude/Codex/OpenCode 的兼容。3. 移动端通过浏览器 / PWA 承载，而不是另开 native app，这样能和桌面端共用同一套前端。

Round 1 共识：三阶段主题为 PR 合并、仓库整理、前端/移动端改造；先把基础问题谈清楚，再进入具体执行。

Round 2：顺序调整与路径规则

SE: 用户最初提的顺序是“先整理仓库、再合并 PR”。但我建议调整为“先合并 PR，再整理仓库”。原因是 PR #1 已经把 `working_dir` 改成相对路径，并补充了缺失依赖；先合并可以让整理阶段直接基于一个更干净的基线，避免整理完再合并时又要重新处理 `server.yaml` 和依赖。

A: 从工程角度看，这个调整是合理的。先合并 PR 后，我们可以在整理阶段一次性把路径规则、目录结构和文档做到位，而不是在两次改动之间反复拉扯。

PM: 我同意这个调整。但需要在文档里明确说明顺序变更的原因，避免后续执行时混淆。

SE: 关于 `working_dir` 的解析规则，我建议：- 绝对路径 → 直接使用。- 相对路径 → 以 **配置文件所在目录**（`Dionysus_CONFIG_DIR`）为基准解析成绝对路径。- 未配置 → 默认 `./workspace`（相对 config dir）。

这样无论开发者从哪里启动 `uvicorn`，工作目录都是确定的。对于 Electron 发布包，`Dionysus_CONFIG_DIR` 指向 `userData/Backend/config`，那么默认 `workspace` 就会落在 `userData/Backend/workspace`，与应用包分离。

A: 同意。但这里有一个产品问题：用户下载安装后，默认工作目录可能是一个空文件夹，用户未必知道该把项目放在哪里。我们需要在 UI 或设置里提供修改工作目录的入口，并在文档中说明。

PM: 对，工作目录必须是用户可配置的，默认路径只是兜底。

Round 2 共识： - 阶段顺序调整为：先合并 PR，再整理仓库，最后做前端/移动端改造。 - `working_dir` 解析规则以 `Dionysus_CONFIG_DIR` 为基准，默认 `./workspace`。 - 工作目录必须支持在 UI/设置中修改。

Round 3：PR 合并与兼容性保障

SE： PR #1 引入了 `is_thinking` 字段，前端新增了 `ThinkingSection`。我需要确认这个字段的兼容性。

A： `is_thinking` 在 Pydantic 模型里应给默认值 `False`，前端协议里也标记为可选？，这样旧 adapter 不传时不会导致解析失败。

FDA： `ThinkingSection` 组件在未收到 thinking 内容时应直接 `return null`，不渲染空区域。PR 里的实现已经这样做了，只需要在合并后验证一下。

PM： 合并后的验证清单我建议包括：1. Kimi/Claude/Codex/OpenCode/CodeBuddy 五个 adapter 都能发起一次简单对话。2. CodeBuddy 的思考过程能正常折叠/展开。3. 旧会话历史不受影响。4. `npm run build` 和 `PyInstaller` 能正常完成。

SE： 测试策略怎么安排？目前后端测试很少，前端几乎没有测试。

A： 合并阶段先为 `CodeBuddyStrategy` 补充单元测试，覆盖 `system/init`、`assistant` 文本/`thinking/tool_use/tool_result`、`result success/error` 等分支。其他 adapter 的回归以 smoke 测试为主。

Round 3 共识： - PR 合并后立刻补全 CodeBuddy 策略单元测试。 - 对所有 adapter 做最小 smoke 验证。 - 前端构建与打包流程必须跑通。

Round 4：视觉方向与移动端策略

FDA： 关于前端视觉方向，用户给出了明确偏好：**明日方舟风格的“类 Fluent Design 设计思想” + 扁平化 2D 美术风格**。

我的理解是： - **类 Fluent Design**：强调分层、半透明/亚克力质感、柔和阴影、清晰的层级关系。 - **扁平化 2D 美术**：避免厚重拟物，使用大色块、简洁几何、锐利或微圆角、干净的图标与排版。 - **明日方舟 UI 气质**：科技感、信息密度适中、功能分区明确、点缀色鲜明（如橙/蓝灰），整体偏冷但不失温度。

我建议把这套风格命名为 **“Dionysus Tech-Flat”**，并建立对应的 Token 体系。

A： 技术实现上，这套风格对 Tailwind 和主题 YAML 提出了要求：颜色需要支持透明叠加、`surface` 层级需要明确、阴影需要统一。我们可以定义 `surface-0` 到 `surface-3` 的层级，配合 `backdrop-blur` 实现亚克力效果。

FDA： 移动端方面，用户要求 **Live2D 功能完整**，但和桌面端主控如何协调可以再议。

我建议的移动端布局方向： - **桌面端 (≥1024px)**：左侧会话列表、中间聊天、右侧 Live2D（主控在聊天区，角色在右陪伴）。 - **平板端 (768px-1023px)**：会话列表可折叠，Live2D 区域缩小或悬浮。 - **手机端 (<768px)**： - 聊天区全屏，作为“主控”。 - Live2D 默认以悬浮球或底部条形式存在，点击展开为半屏/全屏陪伴模式。 - 提供“性能模式”选项：可降低 Live2D 分辨率或暂停渲染。

PM： 这样既能保证 Live2D 功能完整，又不会因为角色常驻而挤压聊天空间。用户后续可以在这个框架下细化交互。

Round 4 共识： - 视觉方向定为 **“类 Fluent Design + 扁平化 2D”**，参考明日方舟 UI 气质。 - 移动端保留完整 Live2D，但以悬浮/抽屉形式与聊天主控区协调。

Round 5：用户反馈与最终确认

PM: 用户针对上一版文档中的待确认事项给出了明确答复，我们逐条讨论一下。

A: 关于“工作目录的 UI 入口”，用户要求在**启动时提示用户选择**，但要有**不再提醒**选项和**保留编辑入口**。这意味着：- 首次启动（或工作目录无效时）弹出选择对话框。- 提供“记住本次选择”或“不再询问”复选框。- 设置页面保留修改入口。- 选择结果持久化到 `agent_settings.json` 或类似位置。

SE: 这个设计是合理的。技术实现上，Electron 启动时可以通过 `dialog.showOpenDialog` 选择目录；浏览器/PWA 端则通过 `<input type="file" webkitdirectory>` 或手动输入路径。后端需要校验目录存在且可写。

FDA: 关于“默认主题”，用户要求**保留当前主题**。也就是说，新的视觉系统应当作为一套新主题或主题变体加入，而不是直接替换现有默认主题。这样老用户升级后不会被强制切换风格。

PM: 同意。我们可以先新增一套“Tech-Flat”主题，默认主题保持不变，用户可在设置中切换。

FDA: 关于“移动端 Live2D 默认状态”，用户要求：**默认收起，一旦用户发送消息给 agent（任务开始）后即展开**。这是一个非常明确的交互规则。

这意味着：- 手机端进入页面时，Live2D 以最小化悬浮球或底部条形式存在。- 用户发送消息后，自动展开到陪伴模式。- Agent 完成任务或用户手动收起后，可回到收起状态。- 应提供手动展开/收起的常驻按钮。

A: 这个规则需要前端状态机支持。可以在 `chatStore` 或 `live2dStore` 中增加 `mobileCompanionExpanded` 状态，并在用户发送消息时触发展开。

PM: 关于“桌面端与移动端的会话同步”，用户要求**需要同步**，且当前设计是**一主机绑定一移动端**，同步这上面的内容即可。

这意味着：- 主机后端作为会话数据的唯一真实来源。- 移动端通过 WebSocket/REST 连接到主机后端，读取同一份会话列表与消息历史。- 不需要复杂的云端多设备同步，只需保证同一局域网或主机暴露地址可访问。- 会话创建、消息发送、状态更新在两端实时同步。

SE: 技术上，现有后端已经基于 SQLite 存储会话，WebSocket 也支持多连接。移动端只需以 `session_id` 加入同一会话即可。需要确保的是：移动端连接时不会因为 `persona_id` 参数而意外创建新会话。

PM: 关于“缓存历史会话”，用户要求**可以缓存，但要可以清除缓存**。这主要影响 PWA 的 service worker 策略：- 可以缓存会话列表和消息历史，提升离线查看体验。- 必须提供“清除本地缓存”的入口，避免占用过多存储。- 清除缓存后，下次联网时从后端重新拉取。

A: 实现上，可以在 IndexedDB 中缓存会话数据，并在设置中提供“清除缓存”按钮。service worker 不直接缓存动态 API，只缓存应用壳和静态资源。

PM: 最后，关于“Electron 入口清理”和“Live2D 源模型归属”这类具体目录调整，用户表示**先由我们梳理文件，然后再决定**。所以这些事项保留在第二阶段，但需要先把梳理结果呈现给用户，而不是直接删除。

Round 5 共识: - 工作目录入口：启动时提示选择，支持“不再提醒”，设置中保留编辑。- 默认主题：保留现有主题，新增 Tech-Flat 主题作为可选。- 移动端 Live2D：默认收起，发送消息后自动展开。- 会话同步：一主机绑定一移动端，主机后端作为唯一数据源，两端实时同步。- 历史缓存：PWA 可缓存历史，但必须提供清除缓存入口。- 目录清理：先梳理呈现，再由用户确认删除/保留。

Round 6：移动端交互细节最终细化

PM: 用户对移动端的几个具体细节给出了最终答复，我们在这里记录下来。

FDA: 关于“移动端 Live2D 展开形态”，用户明确为：**抽屉展开占据屏幕约 80%，从下方对话框向上展开**。这比我们之前设想的“半屏/全屏”更具体。

我的理解是： - 手机端底部常驻一个“角色”触发条/悬浮按钮。 - 点击后，Live2D 区域以一个 **底部抽屉（bottom sheet）** 形式向上滑出，高度约为屏幕高度的 80%。 - 抽屉内部是 Live2D 角色 + 台词气泡，背景做亚克力/半透明模糊处理，与 Fluent Design 方向一致。 - 用户可以通过向下滑动或点击收起按钮关闭抽屉。 - 发送消息触发任务时，抽屉自动展开到 80% 高度；任务结束后用户可手动收起。

A：这个形态在技术上可行。我们可以复用或改造现有的 `MobileCompanionDrawer`，将其高度从全屏改为 80vh，并支持手势拖拽关闭。

SE：关于“移动端工作目录选择实现”，用户指出：**当前移动端不具备使用 agent 的能力，所以还是利用相同局域网或内网穿透的方式连接主机，因此采用和电脑端一样的工作目录选择方案就行。**

这意味着： - 工作目录选择是**主机端**的行为，移动端只负责连接主机。 - 移动端不需要自己选工作目录；它连接主机后，自动使用主机上配置的 adapter 工作目录。 - 工作目录的“启动时提示”主要面向桌面端/Electron；PWA 移动端在连接主机后只展示主机的当前配置。

A：对，移动端本质上是一个“远程控制面板”，所有 Agent 执行都在主机后端完成。工作目录配置、CLI 安装等都在主机端处理。

PM：关于“一主机绑定一移动端的连接方式”，用户确定为：**二维码输入。**

这意味着： - 主机端生成一个包含连接地址（IP/域名 + 端口 + 可选 token）的二维码。 - 移动端通过摄像头扫描二维码，自动填充服务器地址并建立连接。 - 二维码可以由后端 `/api/server/qr` 接口生成（该接口已存在）。 - 绑定后，移动端记住该地址，下次启动自动重连；用户可在设置中解除绑定或重新扫码。

SE：现有后端已经提供了 `/api/server/qr` 接口，可以生成 URL 二维码。我们只需要在前端把这个流程做完整：主机展示二维码 → 移动端扫码 → 移动端保存 base URL → 后续通过该 base URL 连接 WebSocket/REST。

FDA：这样整个移动端的连接体验会非常自然：用户打开电脑端 Dionysus，点击“手机连接”展示二维码，手机一扫即可进入 companion 模式。

Round 6 共识： - 移动端 Live2D 以底部抽屉展开，高度约 80%，向上滑出。 - 工作目录选择仅发生在主机端，移动端复用主机配置。 - 主机与移动端的绑定通过二维码完成。

Round 7：视觉与交互细节最终确认

PM：用户对上一版剩余的几个细化问题也给出了方向，我们逐条确认。

FDA：关于“Tech-Flat 主题的具体配色与字体”，用户指定： - **配色：**蓝色系，工业风，科技感。 - **字体：**由我提供几个方案，但必须考虑版权问题。

基于这个方向，我建议的配色方案如下：

Token	色值	用途
primary	#2563EB	主按钮、强调色、选中状态
primary-hover	#1D4ED8	主按钮悬停
accent	#06B6D4	高亮、进度条、Live2D 气泡点缀
surface-0	#0B1120	最深背景（深色模式）
surface-1	#111827	抽屉/面板背景
surface-2	#1F2937	卡片、输入框背景

Token	色值	用途
surface-3	#374151	分隔线、次级卡片
text-primary	#F9FAFB	主要文字
text-secondary	#9CA3AF	次要文字、时间戳
border	rgba(148, 163, 184, 0.2)	subtle 边框
danger	#EF4444	错误、中断
success	#10B981	成功状态

整体是 深蓝灰工业科技风，用青色作为点睛色，符合明日方舟类 UI 的冷峻科技感。

关于字体，我提供三个候选方案：

1. 系统字体栈（推荐，无版权风险）
2. 使用设备自带字体，如 `-apple-system, BlinkMacSystemFont, "Segoe UI", "PingFang SC", "Microsoft YaHei", sans-serif`。
3. 优点：零加载、零版权、性能最好。
4. 缺点：不同平台显示有差异。
5. **Inter + Noto Sans SC（开源商用）**
6. Inter（SIL Open Font License）负责西文与数字，Noto Sans SC（SIL OFL）负责中文。
7. 优点：风格统一、现代、跨平台一致。
8. 缺点：需要加载字体文件。
9. **Roboto + Noto Sans SC（开源商用）**
10. Roboto（Apache 2.0）负责西文与数字，Noto Sans SC（SIL OFL）负责中文。
11. 优点：工业感强，偏 Android/工具类气质。
12. 缺点：Roboto 中文字体回退时可能略显纤细。

等宽字体建议统一使用 **JetBrains Mono（SIL OFL）** 或系统默认等宽字体。

PM：字体方案我倾向于方案 1（系统字体栈），因为本项目需要频繁展示代码，系统字体在中文渲染上通常更稳定；如果后续需要更强的品牌一致性，可以再切换到方案 2。最终选择权交给用户。

FDA：关于“性能模式触发条件”，用户提出：**如果用户的移动端设备不支持渲染 Live2D，可以手动降级为视频或图片（先保留位置，后续再提供）。**这意味着：- 在移动端设置中提供一个“性能/降级模式”开关。- 当设备不支持 WebGL 或用户主动开启时，Live2D 区域显示角色的静态图片或循环视频（视频资源后续补充）。- 现在先预留 UI 位置和状态管理，图片 fallback 可以先使用角色立绘或模型默认表情截图。- 台词气泡、触摸反馈在降级模式下仍然保留，只是角色不再做 Live2D 动作。

A：这个设计既保证了功能完整性，又给了低性能设备出路。技术上，我们可以在 `live2dStore` 中增加 `renderMode: 'live2d' | 'image' | 'video'`；在 `Live2DViewer` 中根据模式渲染不同内容。

PM：关于“缓存策略细节”，用户明确：**只缓存消息目前。**这意味着：- PWA 在 IndexedDB 中缓存会话列表和消息文本。- 不缓存图片、附件、Live2D 模型等大型资源。- 缓存条目可按会话维度管理，清除缓存时一并清除。- 是否设置上限（如最近 50 条或最近 30 天）可以在执行阶段再细化。

SE：关于“二维码安全”，我提出以下方案：

二维码配对安全方案（建议）： 1. **一次性配对令牌：** 主机生成一个短期有效的配对 token（如 UUID 或 8 位随机字符串），有效期 5 分钟，存储在内存或后端缓存中。 2. **二维码内容：** 编码为 `http(s)://<host>:<port>/?pair_token=<TOKEN>`，而不是长期有效的连接地址。 3. **扫码验证：** 移动端扫描二维码后，向 `/api/pair` 或 WebSocket 握手时提交 `pair_token`。 4. **绑定与鉴权：** 后端验证 token 有效后，为移动端生成一个长期有效的 **设备凭证（device_token）**，移动端后续用该凭证连接；主机端可在设置中查看/撤销已配对设备。 5. **网络要求：** 默认要求移动设备与主机处于同一局域网；若使用内网穿透，token 机制同样适用，但建议额外启用 HTTPS 和访问密码。 6. **撤销配对：** 主机端提供“断开手机连接”按钮，立即使对应 device_token 失效。

这个方案的优点是：即使二维码被拍照截获，5 分钟后也失效；长期凭证由主机控制，可随时撤销。

A： 同意。device_token 可以存储在 `Dionysus_DATA_DIR` 的一个 JSON 文件中，重启后仍然有效。

FDA： 关于“移动端首次连接引导”，用户要求**需要**。我建议设计一个三步引导： 1. **前提确认：** 手机和电脑处于同一 Wi-Fi（或已知主机地址）。 2. **扫码连接：** 展示二维码扫描框，提示用户打开电脑端 Dionysus 的“连接手机”按钮。 3. **完成提示：** 连接成功后显示“已成功绑定主机”，并给出进入聊天/角色模式的入口。

引导页面提供“跳过”和“不再显示”选项，后续可在设置中重新打开。

FDA： 关于“抽屉展开后的交互细节”，用户要求**气泡在角色上方**，具体方案由我来提供。我建议：

移动端 Live2D 抽屉内部布局： - **顶部 8%：** 拖动手柄（一条短横线），提示可下滑收起。 - **上部 20%：** 角色台词气泡，锚定在角色头部上方，最大宽度 80%，最多显示 2 行文字，背景使用 `surface-2` + 亚克力模糊，带小三角指向角色。 - **中部 55%：** Live2D 角色渲染区，角色居中，支持点击头部/身体触发触摸反馈。 - **底部 17%：** 功能条，包含： - 收起抽屉按钮。 - 性能模式 / 降级模式切换。 - 角色切换（如果未来支持）。 - **动效：** - 展开：从底部向上滑入，带弹簧缓动。 - 收起：向下滑出。 - 气泡出现：淡入 + 轻微上飘。

A： 这个布局在 80% 高度的抽屉内比例合理，角色仍然是视觉中心，气泡在上方符合阅读习惯。

Round 7 共识： - Tech-Flat 配色：蓝色系工业科技风，具体 Token 见上表。 - 字体方案：三个候选，默认推荐系统字体栈（无版权风险）。 - 性能模式：手动切换 Live2D / 图片 / 视频，视频资源后续补充。 - 缓存：仅缓存消息文本。 - 二维码安全：采用一次性配对 token + 长期 device_token + 主机可撤销的方案。 - 首次连接引导：三步引导，可跳过、可关闭。 - 抽屉交互：80% 底部抽屉，气泡在角色上方，功能区在底部。

6. 三阶段策划方向（调整后顺序）

第一阶段：PR 合并与兼容性保障

目标： - 将 PR #1 平滑合并到主线。 - 确保现有 Kimi/Claude/Codex/OpenCode 不受影响。 - 为 CodeBuddy 补充基本测试与文档。

核心原则： - **先 review 再合并：** 充分理解 PR 改动范围。 - **向后兼容：** 新增字段必须带有默认值，不影响旧 adapter。 - **回归验证：** 合并后对所有 adapter 做最小 smoke 验证。

重点解决的问题： 1. `is_thinking` 字段的默认值与兼容性。 2. `CodeBuddyStrategy` 的解析鲁棒性。 3. 缺失依赖的补齐（`httpx`、`qrcode[pil]`、`python-multipart`）。 4. 前端 `ThinkingSection` 的空状态处理。

预期产出： - 合并后的主线代码。 - `CodeBuddyStrategy` 单元测试。 - 更新后的 adapter 文档。 - 全量 adapter smoke 验证结论。

第二阶段：仓库整理与开发文档

目标： - 消除所有硬编码本机路径。 - 建立清晰、可维护的项目目录结构。 - 补齐面向开发者的文档。 - 为前端/移动端改造打好基础。

核心原则： - **可移植性：**任何 clone 仓库的开发者，在配置好 CLI 后都能直接运行。 - **隔离性：**源码、配置、运行时数据分离。 - **产品友好：**发布包的默认工作目录落在用户数据区，不在 app bundle 内。

重点解决的问题： 1. 硬编码路径 → 改为相对路径、环境变量或命令行参数。 2. `working_dir` 解析规则 → 以 `Dionysus_CONFIG_DIR` 为基准解析相对路径，默认 `./workspace`。 3. 环境变量命名 → 与 `pydantic-settings` 的 `env_prefix="Dionysus_"` 保持一致。 4. Live2D 资产关系 → 区分“源模型”与“运行时模型”，在文档中说明。 5. Electron 入口 → 确认实际使用入口，清理废弃版本（需先梳理呈现，再由用户确认）。 6. QA/测试脚本 → 改为相对项目根或临时目录，统一输出位置。

预期产出： - 清理后的 `backend/config/server.yaml`。 - 统一的 `workspace/` 默认目录（开发时）/ 用户数据区 `workspace`（发布后）。 - 分类后的 `scripts/qa/`。 - 新增 `plan/` 目录，存放本策划文档。 - `docs/architecture.md`。 - `docs/local_dev.md`。 - `docs/release.md`。 - 更新后的 `README.md`。

第三阶段：前端设计风格重构与移动端适配

目标： - 建立统一的设计系统。 - 完成响应式布局，让手机/平板体验达到可用级别。 - 完善 PWA 能力。

视觉方向： - 类 **Fluent Design + 扁平化 2D 美术风格**，参考明日方舟 UI 气质。 - 强调分层、亚克力/半透明质感、柔和和阴影、清晰的功能分区。 - **蓝色系工业科技风配色**，具体 Token 见“关键决策记录”。 - **字体：**默认推荐系统字体栈（无版权风险），也可选择 Inter/Noto Sans SC 或 Roboto/Noto Sans SC。 - **保留当前默认主题**，新增 Tech-Flat 主题作为可选。

移动端方向： - 走 **响应式 Web + PWA** 路线，维护单一 React 代码库。 - **Live2D 功能完整保留：** - 手机端默认收起（底部触发条/悬浮按钮）。 - 用户发送消息给 agent（任务开始）后，Live2D 以底部抽屉形式向上展开，占据屏幕约 80%。 - 抽屉内部：顶部拖动手柄、角色上方台词气泡、中部 Live2D 角色、底部功能条。 - 任务结束后用户可手动收起；提供常驻展开/收起按钮。 - **性能/降级模式：** - 提供手动切换：Live2D / 静态图片 / 视频循环（视频资源后续补充）。 - 当设备不支持 WebGL 或用户主动开启时，自动/手动降级为图片 fallback。 - **工作目录：**移动端仅连接主机，不独立选择工作目录；工作目录配置在主机端完成。 - **会话同步：**一主机绑定一移动端，主机后端作为唯一数据源，两端通过 WebSocket/REST 实时同步。 - **历史缓存：**PWA 在 IndexedDB 中缓存会话列表和消息文本，设置中提供清除缓存入口；不缓存附件/图片/Live2D 资源。 - **连接方式：** - 主机端展示二维码，移动端扫码绑定。 - 二维码包含一次性 `pair_token`，验证后移动端获得长期 `device_token`。 - 主机端可在设置中撤销已配对设备。 - 绑定后移动端记住地址，下次启动自动重连。 - **首次连接引导：**三步引导页面（确认网络 → 扫码 → 完成），可跳过、可关闭。 - 渐进式推进：先重构设计 Token，再调整桌面响应式骨架，最后打磨移动端细节。

重点解决的问题： 1. Tailwind 硬编码 class 与设计 Token 的对齐。 2. 主题 YAML 与组件代码的“两套皮肤”问题。 3. 移动端布局、触控体验、虚拟键盘适配。 4. Live2D 在移动端的性能与交互方式（80% 底部抽屉 + 气泡在角色上方）。 5. PWA manifest、service worker、离线启动壳的完善。 6. 二维码配对流程与设备凭证管理。 7. 首次连接引导与降级模式预留。

预期产出： - 设计 Token 系统（命名与实现）。 - 响应式布局改造。 - 移动端组件打磨（含 80% 高度 Live2D 抽屉、气泡、功能条）。 - PWA 配置完善（含消息缓存与清除入口）。 - 二维码连接与配对流程。 - 首次连接引导页面。 - `docs/mobile_adaptation.md`。

7. 关键决策记录

决策项	结论	说明
阶段顺序	先合并 PR，再整理仓库，最后做前端/移动端改造	降低 <code>server.yaml</code> 与依赖的反复冲突
工作目录	默认 <code>./workspace</code> （相对 <code>Dionysus_CONFIG_DIR</code> ）	开发时在仓库根目录，发布后在用户数据区
工作目录 UI 入口	启动时提示选择，支持“不再提醒”，设置中保留编辑入口	选择结果持久化到后端配置
路径解析	相对路径以配置文件所在目录为基准解析	避免启动目录不同导致行为不一致
视觉方向	类 Fluent Design + 扁平化 2D，参考明日方舟	新增 Tech-Flat 主题，保留当前默认主题
默认主题	保留现有主题	Tech-Flat 作为可选主题加入
Tech-Flat 配色	蓝色系工业科技风	主色 <code>#2563EB</code> ，点缀 <code>#06B6D4</code> ，深灰背景
字体方案	候选 3 套，默认推荐系统字体栈	详见 Round 7，避免版权问题
移动端 Live2D 默认状态	默认收起，发送消息后自动展开	任务开始触发展开
移动端 Live2D 展开形态	底部抽屉，向上展开，占据屏幕约 80%	顶部手柄、角色上方气泡、底部功能条
移动端工作目录	不复用移动端选择，直接连接主机使用主机配置	移动端是远程控制面板
性能/降级模式	手动切换 Live2D / 图片 / 视频	视频资源后续补充，图片 fallback 可先实现
会话同步	一主机绑定一移动端，主机后端为唯一数据源	两端通过 WebSocket/REST 实时同步
历史缓存	仅缓存会话列表与消息文本，提供清除缓存入口	不缓存附件/图片/Live2D 资源
主机-移动端连接方式	二维码扫描绑定	一次性 <code>pair_token</code> + 长期 <code>device_token</code>
二维码安全	5 分钟有效 <code>pair_token</code> + 主机可控 <code>device_token</code> + 可撤销	详见 Round 7
首次连接引导	需要，三步引导，可跳过/关闭	后续可在设置中重新打开
目录清理	先梳理呈现，再由用户确认删除/保留	尤其是根目录 <code>electron/</code> 、Live2D 源模型
策划文档位置	仓库根目录新建 <code>plan/</code> 文件夹	与 <code>docs/</code> 区分

8. 剩余待决策 / 待细化事项

即使本策划文档获得认可，以下问题仍建议在后续阶段细化：

- 字体最终选择：**在系统字体栈、Inter + Noto Sans SC、Roboto + Noto Sans SC 中确认一套。
- 缓存上限策略：**消息缓存是否有条数/时间上限？
- 视频降级资源：**视频循环素材的制作或来源。
- 二维码 UI 流程：**主机端“连接手机”按钮位置、移动端扫码入口位置。

5. 抽屉内触摸区：在 80% 高度抽屉中，头部/身体触摸区的命中区域是否需要调整？
6. 多移动端绑定：是否只允许一主机绑定一移动端，还是未来可扩展为多移动端？

9. 附录：建议目录结构

```
DionysusC/
├── .github/                # CI 工作流（可选）
├── assets/                 # 源素材（图标、Live2D 源模型等，可选迁移）
│   └── live2d/source/      # kal'tsit、exusia 源模型
├── backend/
│   ├── config/             # 服务端配置（主题、内置角色、server.yaml）
│   │   ├── personas/builtin/
│   │   ├── themes/
│   │   └── server.yaml
│   ├── dionysus_server/    # FastAPI 后端源码
│   │   ├── agent_adapters/
│   │   ├── persona/
│   │   ├── session/
│   │   ├── websocket/
│   │   ├── config.py
│   │   ├── main.py
│   │   ├── models.py
│   │   └── theme_manager.py
│   ├── tests/              # 后端测试
│   ├── dionysus_server.spec
│   ├── electron_entry.py
│   ├── pyproject.toml
│   └── requirements.txt
├── docs/
│   ├── architecture.md
│   ├── local_dev.md
│   ├── release.md
│   ├── mobile_adaptation.md
│   ├── user_guide.md
│   └── screenshots/
├── frontend/
│   ├── electron/           # Electron 主进程与 preload（确认后的入口）
│   ├── public/             # 静态资源、PWA icons
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── lib/
│   │   ├── services/
│   │   ├── stores/
│   │   ├── styles/tokens.ts
│   │   ├── types/
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── tests/              # Playwright / smoke 测试
│   ├── index.html
│   ├── package.json
│   └── vite.config.ts
├── plan/                   # 新增：策划文档与阶段计划
│   └── phase_overview.md
├── scripts/
│   ├── dev.sh              # 一键启动开发环境（可选）
│   ├── launcher.py
│   └── qa/                 # QA 脚本
```

```
├── workspace/                # 默认 adapter 工作目录 (开发时, .gitkeep)
├── .env.example
├── .gitignore
├── Makefile
└── README.md
```

10. 结语

本文档通过七轮多角色讨论，并结合用户反馈，完成了以下工作：

1. 梳理了仓库当前的主要问题（硬编码路径、目录结构、文档缺口、移动端现状）。
2. 明确了 PR #1 的改动范围与合并关注点。
3. 将原定顺序调整为：**先合并 PR，再整理仓库，最后做前端/移动端改造。**
4. 确定了视觉方向（类 Fluent Design + 扁平化 2D，参考明日方舟），并决定保留当前默认主题。
5. 确定了 Tech-Flat 主题的蓝色系工业科技风配色与三套字体候选方案。
6. 确定了移动端策略（响应式 Web + PWA，完整保留 Live2D）。
7. 细化了移动端 Live2D 的交互形态（80% 底部抽屉，气泡在角色上方，底部功能条）。
8. 确定了工作目录入口规则（启动时提示 + 不再提醒 + 设置编辑）。
9. 确定了会话同步与历史缓存策略（一主机一移动端、仅缓存消息、可清除缓存）。
10. 确定了主机与移动端的绑定方式（二维码扫描 + 一次性 pair_token + 长期 device_token）。
11. 确定了首次连接引导与性能/降级模式预留方案。
12. 确定了策划文档存放位置（仓库根目录 `plan/`）。
13. 列出了仍需在后续阶段细化的 6 个讨论事项。

下一步建议：用户审阅并确认本文档后，再基于本文档分别制定每个阶段的详细执行计划。